

Fase 1 – pruebas de queries:

The screenshot shows the pgAdmin 4 interface with a PostgreSQL 18 server named 'nba_project'. The 'Query' tab is active, displaying a SQL query that counts rows for various tables. The 'Data Output' tab shows the results of the query, which includes 11 rows of data. A green message at the bottom indicates 'Successfully run. Total query runtime: 96 msec. 11 rows affected.'

```
1 -- Conteos básicos después de ejecutar la carga.
2 SELECT 'team' AS table_name, COUNT(*) AS rows FROM team
3 UNION ALL SELECT 'season', COUNT(*) FROM season
4 UNION ALL SELECT 'player', COUNT(*) FROM player
5 UNION ALL SELECT 'game', COUNT(*) FROM game
6 UNION ALL SELECT 'official', COUNT(*) FROM official
7 UNION ALL SELECT 'game_official', COUNT(*) FROM game_official
8 UNION ALL SELECT 'team_salary', COUNT(*) FROM team_salary
9 UNION ALL SELECT 'player_salary', COUNT(*) FROM player_salary
10 UNION ALL SELECT 'draft_selection', COUNT(*) FROM draft_selection
11 UNION ALL SELECT 'team_history', COUNT(*) FROM team_history
12 UNION ALL SELECT 'player_season_stat', COUNT(*) FROM player_season_stat
```

table_name	rows
draft_selection	7890
game	62379
game_official	65158
official	143
player	9035
player_salary	1292
player_season_stat	540
season	80
team	45
team_history	60
team_salary	180

The screenshot shows the pgAdmin 4 interface with a PostgreSQL 18 server named 'nba_project'. The 'Query' tab is active, displaying a SQL query that checks for duplicate game IDs. The 'Data Output' tab shows the results of the query, which includes 0 rows of data. A green message at the bottom indicates 'Successfully run. Total query runtime: 87 msec. 0 rows affected.'

```
10 UNION ALL SELECT 'draft_selection', COUNT(*) FROM draft_selection
11 UNION ALL SELECT 'team_history', COUNT(*) FROM team_history
12 UNION ALL SELECT 'player_season_stat', COUNT(*) FROM player_season_stat
13 ORDER BY table_name;
14
15 -- No deben existir partidos con el mismo identificador.
16 SELECT game_id, COUNT(*)
17 FROM game
18 GROUP BY game_id
19 HAVING COUNT(*) > 1;
20
21 -- No deben existir relaciones huérfanas de árbitros.
```

game_id	count
---------	-------

Control de duplicados de GAME_ID: 0 filas, sin duplicados

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (3)
 - lab04
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - nba_project
 - Casts
 - Catalogs (2)
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - postgres
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications

verification.sql X

nba_project/postgres@PostgreSQL 18

Query Query History

```

18 GROUP BY game_id
19 HAVING COUNT(*) > 1;
20
21 -- No deben existir relaciones huérfanas de árbitros.
22 SELECT go.*
23 FROM game_official go
24 LEFT JOIN game g ON g.game_id = go.game_id
25 LEFT JOIN official o ON o.official_id = go.official_id
26 WHERE g.game_id IS NULL OR o.official_id IS NULL;
27
28 -- Temporadas mínimas requeridas para el proyecto.
29 SELECT season_id, COUNT(*) AS games

```

Data Output Messages Notifications

game_id	official_id
[PK] character varying (20)	[PK] bigint

Successfully run. Total query runtime: 74 msec. 0 rows affected.

Total rows: 0 Query complete 00:00:00.074 CRLF Ln 21, Col 1

7:51 PM 8/27/2026

Control de relaciones huérfanas de árbitros

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (3)
 - lab04
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - nba_project
 - Casts
 - Catalogs (2)
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - postgres
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications

verification.sql X

nba_project/postgres@PostgreSQL 18

Query Query History

```

26 WHERE g.game_id IS NULL OR o.official_id IS NULL;
27
28 -- Temporadas mínimas requeridas para el proyecto.
29 SELECT season_id, COUNT(*) AS games
30 FROM game
31 WHERE CAST(SUBSTRING(season_id FROM 1 FOR 4) AS INTEGER) BETWEEN 2015 AND 2020
32 GROUP BY season_id
33 ORDER BY season_id;
34
35 -- Deben existir las seis temporadas mínimas. debe devolver cero filas.
36 WITH requeridas AS (SELECT generate_series(2015, 2020) AS start_year)
37 SELECT r.start_year

```

Data Output Messages Notifications

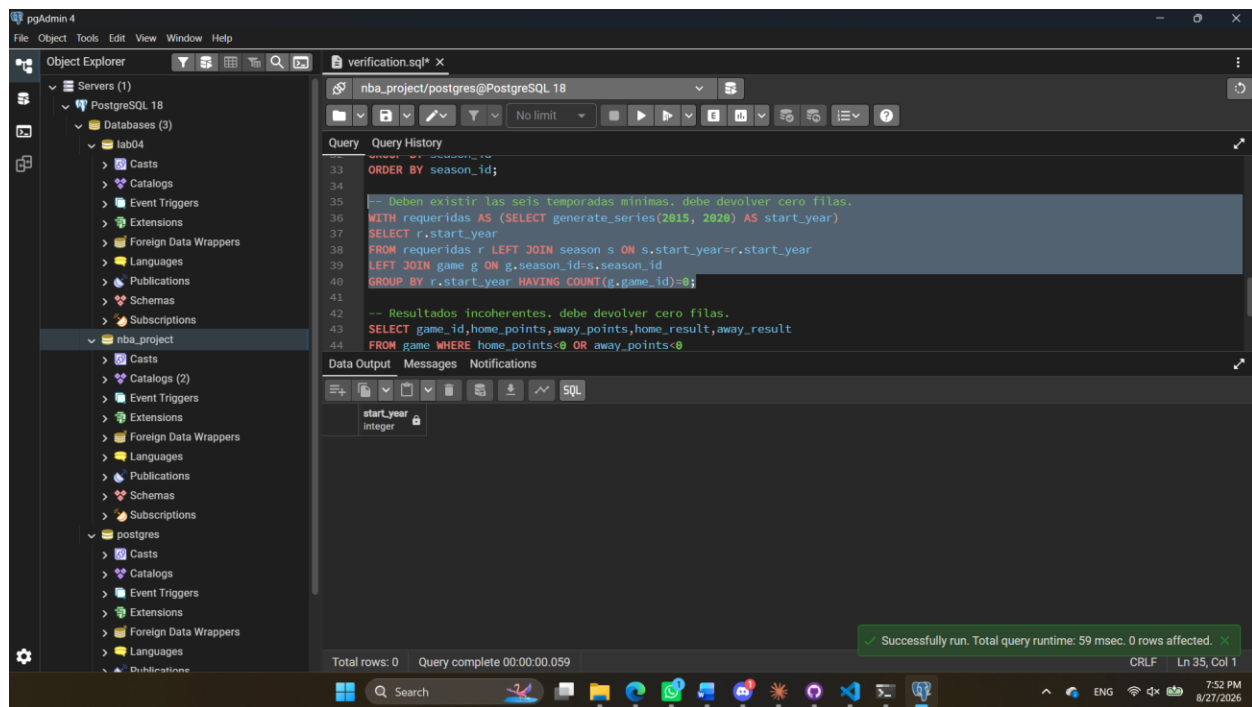
season_id	games
character varying (7)	bigint
1 2015-16	1230
2 2016-17	1230
3 2017-18	1230
4 2018-19	1230
5 2019-20	1059
6 2020-21	1080

Showing rows: 1 to 6 Page No: 1 of 1

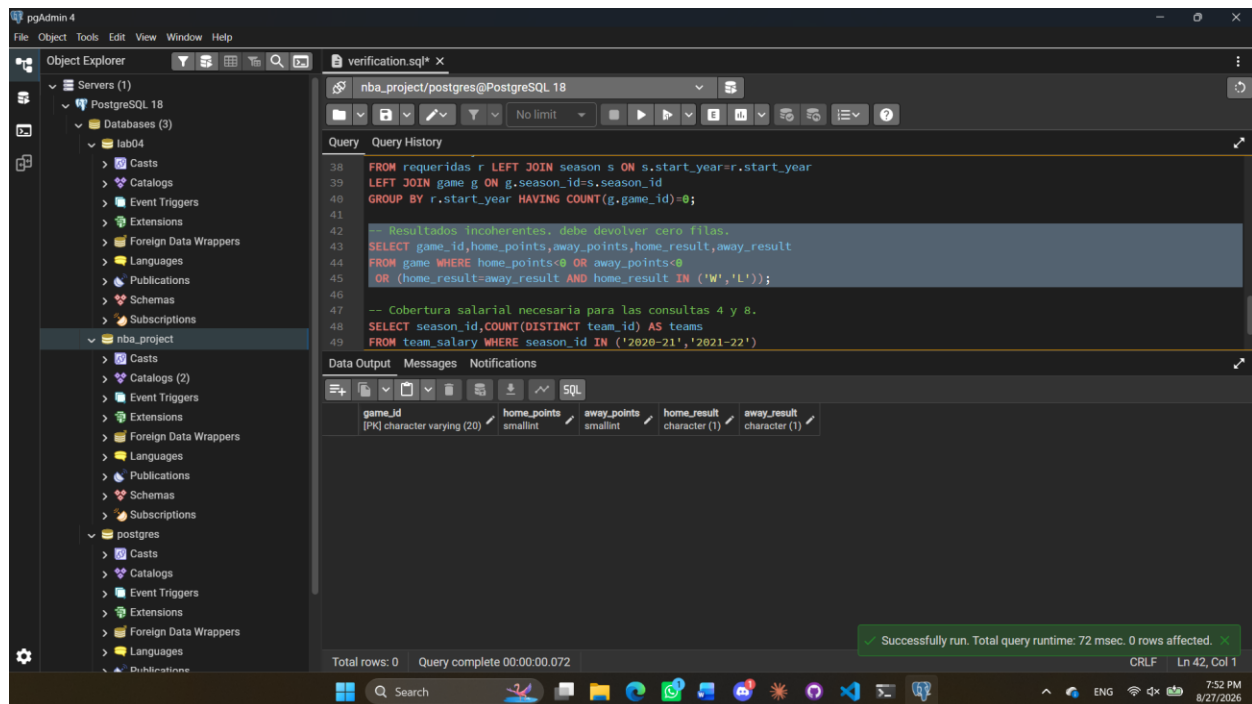
Successfully run. Total query runtime: 72 msec. 6 rows affected.

Total rows: 6 Query complete 00:00:00.072 CRLF Ln 28, Col 1

7:51 PM 8/27/2026



control de temporadas mínimas requeridas



control de resultados incoherentes

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (3)
 - lab04
 - Cast
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - nba_project
 - Cast
 - Catalogs (2)
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Subscriptions

verification.sql*

nba_project/postgres@PostgreSQL 18

No limit

Query Query History

```
41
42 -- Resultados incoherentes. debe devolver cero filas.
43 SELECT game_id,home_points,away_points,home_result,away_result
44 FROM game WHERE home_points<0 OR away_points<0
45 OR (home_result=away_result AND home_result IN ('W','L'));
46
47 -- Cobertura salarial necesaria para las consultas 4 y 8.
48 SELECT season_id,COUNT(DISTINCT team_id) AS teams
49 FROM team_salary WHERE season_id IN ('2020-21','2021-22')
50 GROUP BY season_id ORDER BY season_id;
51
52
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

season_id	teams
2020-21	30
2021-22	30

Total rows: 2 Query complete 00:00:00.074

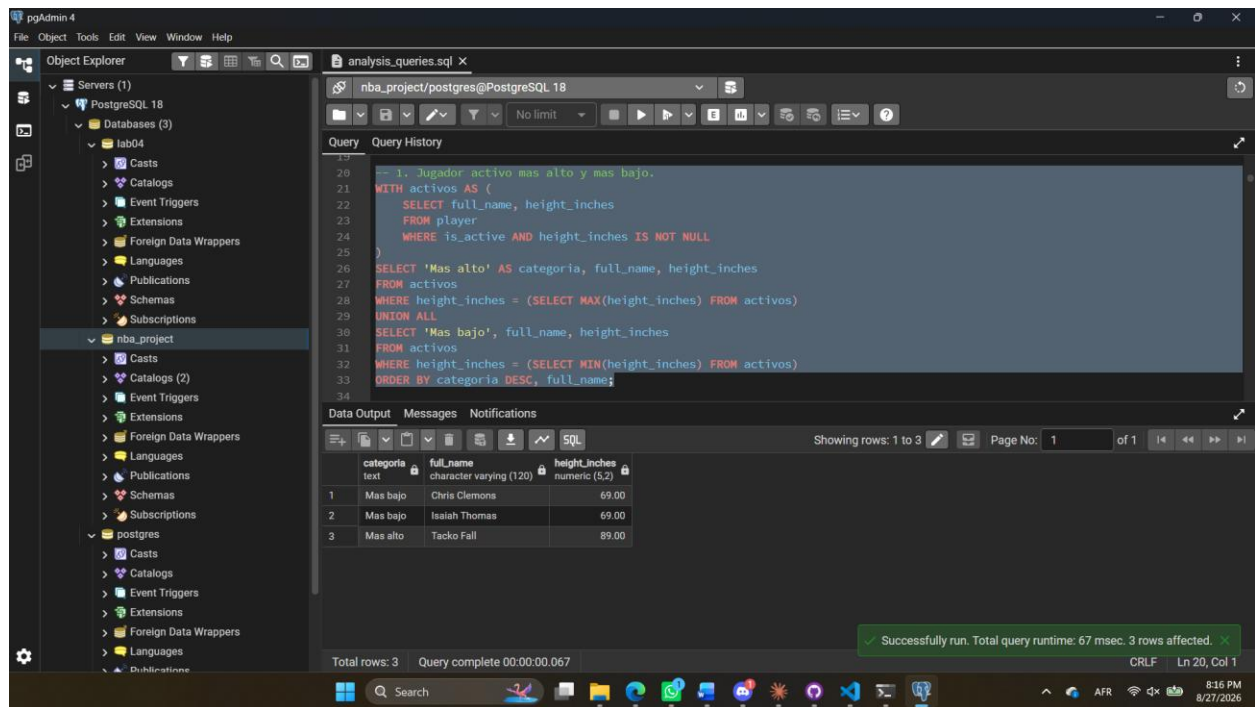
Successfully run. Total query runtime: 74 msec. 2 rows affected.

CRLF Ln 47, Col 1

7:52 PM 8/21/2026

Fase 2 – Queries

1. ¿Quién es el jugador activo más alto? ¿Y el más bajo?



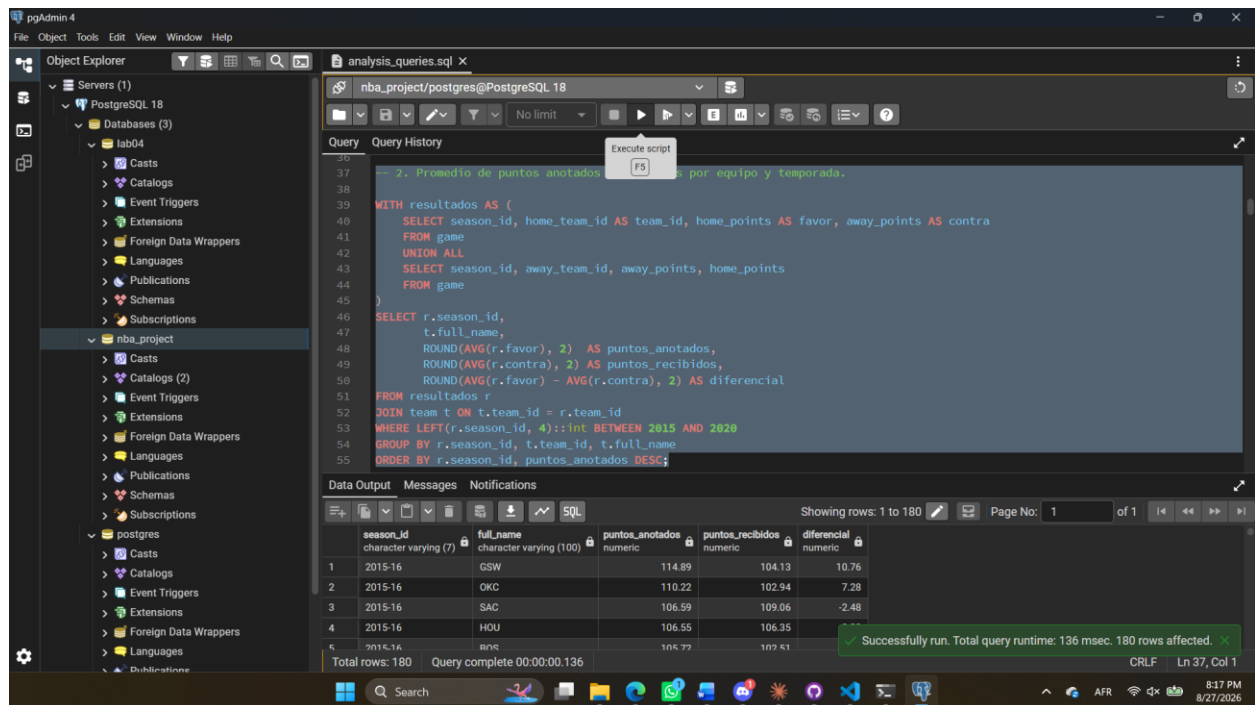
The screenshot shows the pgAdmin 4 interface with a PostgreSQL 18 database named 'nba_project'. The 'Object Explorer' on the left shows the database structure. The 'Query' tab is active, displaying a SQL query that identifies the tallest and shortest active players. The 'Data Output' tab shows the results of the query.

```
1. Jugador activo mas alto y mas bajo.
WITH activos AS (
  SELECT full_name, height_inches
  FROM player
  WHERE is_active AND height_inches IS NOT NULL
)
SELECT 'Mas alto' AS categoria, full_name, height_inches
FROM activos
WHERE height_inches = (SELECT MAX(height_inches) FROM activos)
UNION ALL
SELECT 'Mas bajo', full_name, height_inches
FROM activos
WHERE height_inches = (SELECT MIN(height_inches) FROM activos)
ORDER BY categoria DESC, full_name;
```

categoria	full_name	height_inches
Mas bajo	Chris Clemons	69.00
Mas bajo	Isaiah Thomas	69.00
Mas alto	Tacko Fall	89.00

Successfully run. Total query runtime: 67 msec. 3 rows affected.

2. ¿Cuál fue el promedio de puntos anotados y recibidos por cada equipo en cada una de las temporadas relevantes?



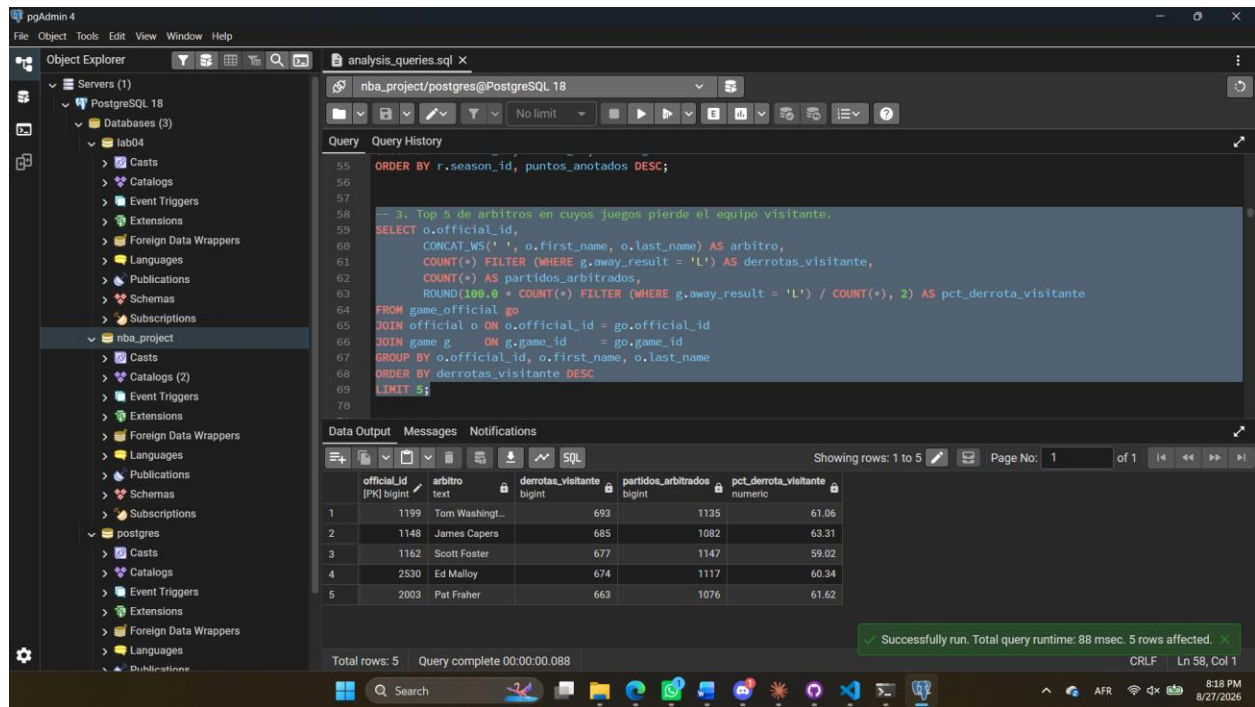
The screenshot shows the pgAdmin 4 interface with a PostgreSQL 18 database named 'nba_project'. The 'Object Explorer' on the left shows the database structure. The 'Query' tab is active, displaying a SQL query that calculates the average points scored and received by each team in each season. The 'Data Output' tab shows the results of the query.

```
-- 2. Promedio de puntos anotados y recibidos por equipo y temporada.
WITH resultados AS (
  SELECT season_id, home_team_id AS team_id, home_points AS favor, away_points AS contra
  FROM game
  UNION ALL
  SELECT season_id, away_team_id, away_points, home_points
  FROM game
)
SELECT r.season_id,
       t.full_name,
       ROUND(AVG(r.favor), 2) AS puntos_anotados,
       ROUND(AVG(r.contra), 2) AS puntos_recibidos,
       ROUND(AVG(r.favor) - AVG(r.contra), 2) AS diferencial
FROM resultados r
JOIN team t ON t.team_id = r.team_id
WHERE LEFT(r.season_id, 4)::int BETWEEN 2015 AND 2020
GROUP BY r.season_id, t.team_id, t.full_name
ORDER BY r.season_id, puntos_anotados DESC;
```

season_id	full_name	puntos_anotados	puntos_recibidos	diferencial
2015-16	GSW	114.89	104.13	10.76
2015-16	OKC	110.22	102.94	7.28
2015-16	SAC	106.59	109.06	-2.48
2015-16	HOU	106.55	106.35	0.20
2015-16	BKN	104.72	102.61	12.11

Successfully run. Total query runtime: 136 msec. 180 rows affected.

3. Muestre el top 5 de árbitros en cuyos juegos pitados el equipo visitante pierde.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the 'Object Explorer' with a tree view of the database structure. The 'nba_project' database is selected, showing its schemas and tables. The main pane shows a SQL query in the 'Query History' tab. The query is as follows:

```
55 ORDER BY r.season_id, puntos_ anotados DESC;
56
57
58 -- 3. Top 5 de árbitros en cuyos juegos pierde el equipo visitante.
59 SELECT o.official_id,
60        CONCAT_WS(' ', o.first_name, o.last_name) AS arbitro,
61        COUNT(*) FILTER (WHERE g.away_result = 'L') AS derrotas_visitante,
62        COUNT(*) AS partidos_arbitrados,
63        ROUND(100.0 * COUNT(*) FILTER (WHERE g.away_result = 'L') / COUNT(*), 2) AS pct_derrota_visitante
64 FROM game_official go
65 JOIN official o ON o.official_id = go.official_id
66 JOIN game g ON g.game_id = go.game_id
67 GROUP BY o.official_id, o.first_name, o.last_name
68 ORDER BY derrotas_visitante DESC
69 LIMIT 5;
```

The 'Data Output' tab shows the results of the query, displaying 5 rows. The columns are: officialId (PK), arbitro, derrotas_visitante, partidos_arbitrados, and pct_derrota_visitante. The results are as follows:

officialId (PK)	arbitro	derrotas_visitante	partidos_arbitrados	pct_derrota_visitante
1	1199 Tom Washington	693	1135	61.06
2	1148 James Capers	685	1082	63.31
3	1162 Scott Foster	677	1147	59.02
4	2530 Ed Malloy	674	1117	60.34
5	2003 Pat Fraher	663	1076	61.62

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.088'. A green message box states 'Successfully run. Total query runtime: 88 msec. 5 rows affected.'

4. ¿Qué equipo maneja los salarios más altos actualmente (última temporada)? Compare esa información ahora con los equipos con los jugadores más valiosos, ¿existe alguna relación? ¿Los equipos con salarios más altos tienen a los jugadores más valiosos?

pgAdmin 4

Object Explorer

Servers (1)

- PostgreSQL 18
 - Databases (3)
 - lab04
 - nba_project
 - postgres

Query

```
-- 4a. Equipo con la nomina mas alta en la ultima temporada, comparado con los equipos que tienen los jugadores mas valiosos.
WITH nomina AS (
  SELECT ts.team_id,
         t.full_name,
         ts.total_salary,
         RANK() OVER (ORDER BY ts.total_salary DESC) AS rank_nomina
  FROM team_salary ts
  JOIN team t ON t.team_id = ts.team_id
  WHERE ts.season_id = '2020-21'
),
valor AS (
  SELECT ps.team_id,
         MAX(ps.salary_value) AS salario_mas_alto,
         ROUND(AVG(p.pie), 4) AS pie_promedio,
         ROUND(MAX(p.pie), 4) AS pie_estrella,
         RANK() OVER (ORDER BY MAX(p.pie) DESC NULLS LAST) AS rank_valor
  FROM player_salary ps
  JOIN player p ON p.player_id = ps.player_id
  WHERE ps.season_id = '2020-21'
  GROUP BY ps.team_id
)
SELECT n.full_name, n.total_salary, n.rank_nomina, n.salario_mas_alto, n.pie_estrella, n.rank_valor, n.brecha
FROM nomina n
JOIN valor v ON n.team_id = v.team_id
```

Data Output

	full_name character varying (100)	total_salary numeric (15,2)	rank_nomina bigint	salario_mas_alto numeric	pie_estrella numeric	rank_valor bigint	brecha bigint
1	GSW	170467299.00	1	43006362.00	0.1760	7	-6
2	BKN	168854452.00	2	41254920.00	0.1700	9	-7
3	PHI	147401509.00	3	34358850.00	0.2110	1	2
4	LAC	138604159.00	4	35450412.00	0.1690	10	-6
5	LAL	138326578.00	5	39219566.00	0.1940	4	1
6	UTA	136048278.00	6	34502132.00	0.1580	15	-9
7	MIL	135289205.00	7	33051724.00	0.1960		

Total rows: 30 Query complete 00:00:00.060

Successfully run. Total query runtime: 60 msec. 30 rows affected.

pgAdmin 4

Object Explorer

Servers (1)

- PostgreSQL 18
 - Databases (3)
 - lab04
 - nba_project
 - postgres

Query

```
-- 4b. Medida objetiva de la relacion entre gasto y talento.
WITH n AS (
  SELECT team_id, total_salary FROM team_salary WHERE season_id = '2020-21'
),
v AS (
  SELECT ps.team_id, MAX(p.pie) AS pie_estrella
  FROM player_salary ps
  LEFT JOIN player p ON p.player_id = ps.player_id
  WHERE ps.season_id = '2020-21'
  GROUP BY ps.team_id
)
SELECT ROUND(CORR(n.total_salary, v.pie_estrella)::numeric, 4) AS correlacion_nomina_valor,
       COUNT(*) AS equipos_comparados
FROM n JOIN v ON v.team_id = n.team_id
WHERE v.pie_estrella IS NOT NULL;
```

Data Output

	correlacion_nomina_valor numeric	equipos_comparados bigint
1	0.4791	30

Total rows: 1 Query complete 00:00:00.058

Successfully run. Total query runtime: 58 msec. 1 rows affected.

5. ¿Cuál fue la temporada con más partidos en la historia de la NBA? ¿Cuál fue la temporada que más se ha prolongado en fechas?

pgAdmin 4

Object Explorer

Servers (1)

- PostgreSQL 18
 - Databases (3)
 - lab04
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - nba_project
 - Casts
 - Catalogs (2)
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Subscriptions
 - postgres
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Subscriptions

analysis_queries.sql*

nba_project/postgres@PostgreSQL 18

Query

```

-- 5a. Temporada con mas partidos en la historia de la NBA.
WITH s AS (
  SELECT season_id,
         COUNT(*) AS partidos,
         MIN(game_date) AS inicio,
         MAX(game_date) AS fin,
         MAX(game_date) - MIN(game_date) AS dias
  FROM game
  GROUP BY season_id
)
SELECT season_id, partidos, inicio, fin, dias
FROM s
WHERE partidos = (SELECT MAX(partidos) FROM s)
ORDER BY season_id;

```

Data Output

season_id	partidos	inicio	fin	dias
1	1230	2004-11-...	2005-04-...	169
2	1230	2005-11-...	2006-04-...	169
3	1230	2006-10-...	2007-04-...	169
4	1230	2007-10-...	2008-04-...	169
5	1230	2008-10-...	2009-04-...	169
6	1230	2009-10-...	2010-04-...	169
7	1230	2010-10-...	2011-04-...	169
8	1230	2013-10-...	2014-04-...	169

Total rows: 13 Query complete 00:00:00.083

Successfully run. Total query runtime: 83 msec. 13 rows affected.

pgAdmin 4

Object Explorer

Servers (1)

- PostgreSQL 18
 - Databases (3)
 - lab04
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - nba_project
 - Casts
 - Catalogs (2)
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Subscriptions
 - postgres
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Subscriptions

analysis_queries.sql*

nba_project/postgres@PostgreSQL 18

Query

```

-- 5b. Temporada que mas se prolongo en fechas.
WITH s AS (
  SELECT season_id,
         COUNT(*) AS partidos,
         MIN(game_date) AS inicio,
         MAX(game_date) AS fin,
         MAX(game_date) - MIN(game_date) AS dias
  FROM game
  GROUP BY season_id
)
SELECT season_id, partidos, inicio, fin, dias
FROM s
ORDER BY dias DESC
LIMIT 5;

```

Data Output

season_id	partidos	inicio	fin	dias
1	1059	2019-10-...	2020-08-...	297
2	902	1978-10-...	1979-04-...	177
3	1230	2018-10-...	2019-04-...	176
4	1230	2017-10-...	2018-04-...	176
5	902	1977-10-...	1978-04-...	173

Total rows: 5 Query complete 00:00:00.075

Successfully run. Total query runtime: 75 msec. 5 rows affected.

6. ¿Cuál es el equipo que más diferencia de puntos a favor obtuvo en promedio por partido para la temporada 2017? ¿Y para la temporada 2018?

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure with 'nba_project' selected. The central pane shows a SQL query in the 'analysis_queries.sql' editor. The query is as follows:

```

6. Equipo con mayor diferencia de puntos a favor por partido en las temporadas 2017-18 y 2018-19.

WITH margenes AS (
    SELECT season_id, home_team_id AS team_id, home_points - away_points AS margen
    FROM game
    UNION ALL
    SELECT season_id, away_team_id, away_points - home_points
    FROM game
),
promedios AS (
    SELECT season_id,
           team_id,
           ROUND(AVG(margen), 2) AS margen_promedio,
           COUNT(*) AS partidos,
           RANK() OVER (PARTITION BY season_id ORDER BY AVG(margen) DESC) AS posicion
    FROM margenes
    WHERE season_id IN ('2017-18', '2018-19')
    GROUP BY season_id, team_id
)
SELECT * FROM promedios;

```

The bottom pane displays the results of the query in a table format. The table has 6 rows and 4 columns: season_id, full_name, margen_promedio, and partidos.

season_id	full_name	margen_promedio	partidos
1 2017-18	HOU	8.48	82
2 2017-18	TOR	7.78	82
3 2017-18	GSW	5.98	82
4 2018-19	MIL	8.87	82
5 2018-19	GSW	6.46	82
6 2018-19	TOR	6.09	82

A status bar at the bottom indicates: 'Successfully run. Total query runtime: 66 msec. 6 rows affected.'

7. ¿Quién es el jugador más valioso del draft1 del 2018 hoy en día (última temporada)?

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure with 'nba_project' selected. The central pane shows a SQL query in the 'analysis_queries.sql' editor. The query is as follows:

```

-- 7. Jugador mas valioso del draft 2018 en la ultima temporada.

SELECT d.player_name,
       d.overall_pick,
       t.full_name AS equipo_que_lo_selecciono,
       p.pie AS pie_carrera,
       s.points_per_game,
       s.assists_per_game,
       s.rebounds_per_game,
       s.season_id AS temporada_api
FROM draft_selection d
JOIN player p ON p.player_id = d.player_id
LEFT JOIN team t ON t.team_id = d.team_id
LEFT JOIN player_season_stat s
    ON s.player_id = p.player_id
   AND s.season_id = (SELECT MAX(season_id) FROM player_season_stat)
WHERE d.draft_year = 2018
ORDER BY s.points_per_game DESC NULLS LAST, p.pie DESC NULLS LAST;

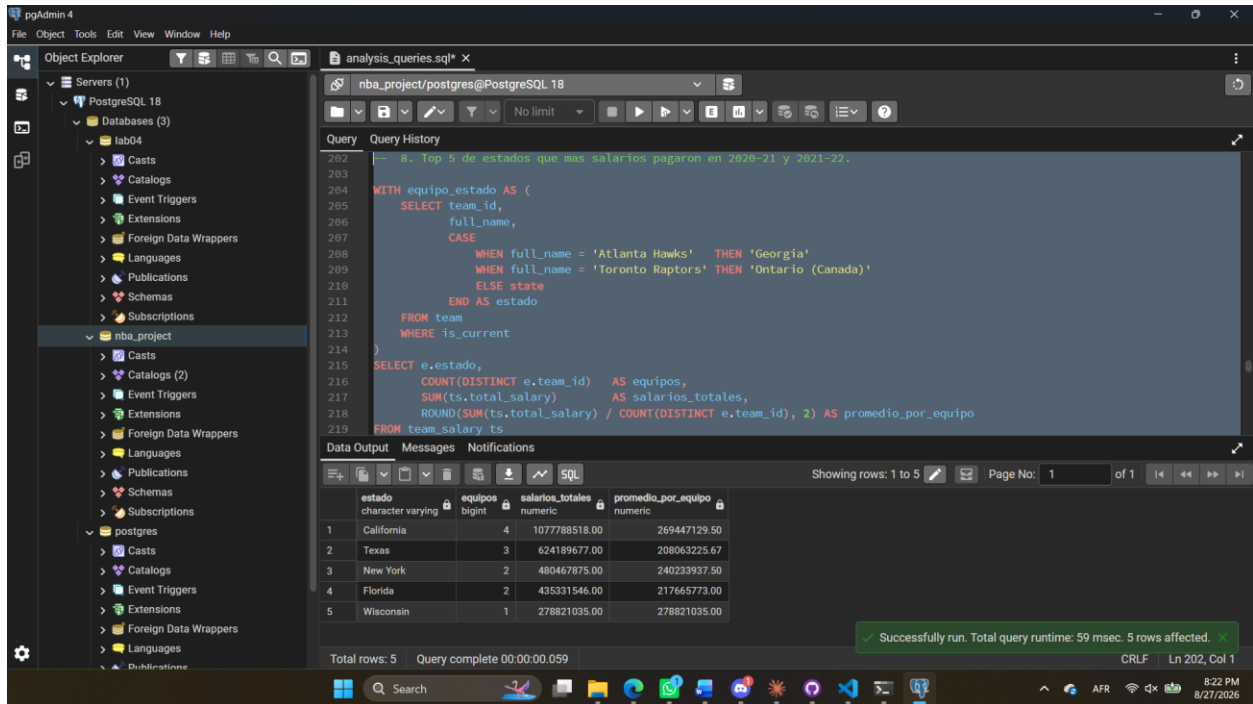
```

The bottom pane displays the results of the query in a table format. The table has 10 rows and 8 columns: player_name, overall_pick, equipo_que_lo_selecciono, pie_carrera, points_per_game, assists_per_game, rebounds_per_game, and temporada_api.

player_name	overall_pick	equipo_que_lo_selecciono	pie_carrera	points_per_game	assists_per_game	rebounds_per_game	temporada_api
1 Luka Doncic	3	ATL	0.1870	27.70	8.60	8.00	2020-21
2 Trae Young	5	DAL	0.1510	25.30	9.40	3.90	2020-21
3 Collin Sexton	8	CLE	0.1120	24.30	4.40	3.10	2020-21
4 Shai Gilgeous-Alexander	11	CHA	0.1460	23.70	5.90	4.70	2020-21
5 Michael Porter Jr.	14	DEN	0.1160	19.00	1.10	7.30	2020-21
6 Khry Thomas	38	PHI	[null]				

A status bar at the bottom indicates: 'Successfully run. Total query runtime: 57 msec. 10 rows affected.'

8. Calcule el top 5 de los estados que más salarios pagaron durante las temporadas 2020/2021 y 2021/2022.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'nba_project' database. The main window shows a SQL query in the 'Query History' tab, which is a query to find the top 5 states by total salary. The query is as follows:

```
202  e. Top 5 de estados que mas salarios pagaron en 2020-21 y 2021-22.
203
204  WITH equipo_estado AS (
205      SELECT team_id,
206             full_name,
207             CASE
208                 WHEN full_name = 'Atlanta Hawks' THEN 'Georgia'
209                 WHEN full_name = 'Toronto Raptors' THEN 'Ontario (Canada)'
210                 ELSE state
211             END AS estado
212      FROM team
213      WHERE is_current
214  )
215  SELECT e.estado,
216         COUNT(DISTINCT e.team_id) AS equipos,
217         SUM(ts.total_salary) AS salarios_totales,
218         ROUND(SUM(ts.total_salary) / COUNT(DISTINCT e.team_id), 2) AS promedio_por_equipo
219  FROM team_salary ts
```

The 'Data Output' tab shows the results of the query, displaying a table with 5 rows and 4 columns: estado, equipos, salarios_totales, and promedio_por_equipo. The results are as follows:

	estado	equipos	salarios_totales	promedio_por_equipo
1	California	4	1077788518.00	269447129.50
2	Texas	3	624189677.00	208063225.67
3	New York	2	480467875.00	240233937.50
4	Florida	2	435331546.00	217665773.00
5	Wisconsin	1	278821035.00	278821035.00

The status bar at the bottom indicates that the query was successfully run, with a total query runtime of 59 msec and 5 rows affected. The status bar also shows the current row (Ln 202, Col 1) and the time (8:22 PM 8/27/2026).